# 第 5 章 Agent 与 Harness：让模型真的把事做成

> 本章目标：分清"聊天"和"干活"的差别，看懂 Agent 的循环是怎么转的，理解 harness 为什么比模型选型更影响结果，并学会一套能复现的工程规矩。

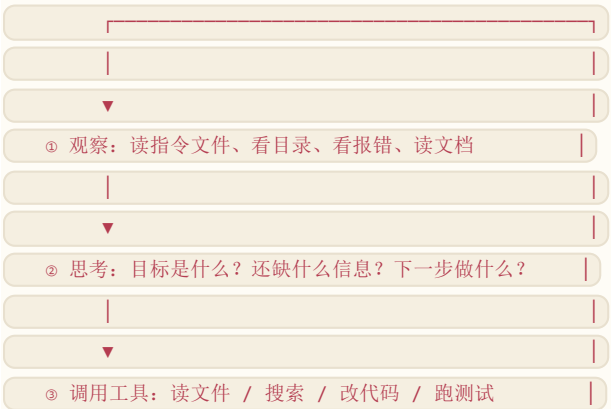
## 5.1 Agent 是什么：五个零件

一个 Agent 不是"更聪明的聊天框"，而是下面五样东西的组合：

| 零件 | 作用 | 缺了会怎样 |
| --- | --- | --- |
| 模型（LLM） | 读情况、做判断、写下一步 | 没有决策者 |
| 工具 | 读写文件、跑命令、搜索、操作界面 | 只能说，不能做 |
| 循环 | 做完一步看结果，再决定下一步 | 一问一答，无法多步 |
| 记忆 | 上下文、指令文件、任务记录 | 每次从零开始，反复踩同一个坑 |
| 权限 | 什么能做、什么要先问 | 一次误操作就能造成真实损失 |

前四个决定它"能做什么"，第五个决定它"被允许做什么"。新手最容易忽略的是第五个——而它恰恰是聊天与 Agent 的分水岭。

## 5.2 Agent 循环：观察 → 思考 → 调用工具 → 读结果






关键在 ④ 和 ⑤：**检查是循环的一部分，不是循环之外的可选项**。模型不需要"猜自己做得对不对"，它可以让程序告诉它——跑测试、编译、打开页面看、`git diff` 看改了什么。

没有 ④ 的 Agent 是什么样：改完就说"已完成"，而实际上文件里多个逗号、测试没跑、页面打不开。有了④，错误在下一步就被发现，而不是在演示现场被发现。

> 要点：Agent 的可靠性不来自"模型更聪明"，来自"每一步都有客观反馈"。你能给它造的反馈越多（测试、构建、日志、截图），它越靠谱。

## 5.3 工具调用到底怎么发生的

模型本身不能碰你的电脑。它只能**输出文本**。工具调用（function calling）是一套约定好的格式：

```
① 程序把工具的说明书交给模型
   {"name": "read_file", "description": "读取文件内容",
    "parameters": {"path": "string，绝对路径"}}

② 模型决定要用它，输出的不是自然语言，而是一段结构化参数
   {"tool": "read_file", "args": {"path": "E:/proj/src/app.js"}}

③ 程序解析这段参数，自己去执行（真正碰磁盘的是程序，不是模型）

④ 执行结果（内容，或报错原文）被塞回上下文

⑤ 模型看到结果，继续下一步
```

由此可以推出几件很重要的事：

- 模型**只能提出**调用。执行、拦截、报错都由程序负责。所以"权限"是在 ③ 这一步真正生效的。
- 参数是模型**生成**的，会编。路径写错、字段名写错、传个不存在的参数，都是常见故障。工具返回的报错原文比模型的猜测可靠得多。

- 工具的**说明书质量**直接决定成败。描述含糊的工具，模型就会乱用。
- 工具返回的内容会成为上下文的一部分，也就会**占用 token、可能带来污染**。

在工具调用之上，有几层常见的扩展：

| 机制 | 本质 | 解决什么 |
| --- | --- | --- |
| MCP | 一套标准协议，让外部服务按统一格式暴露工具 | 不用为每个产品各写一套接入 |
| SKILL | 打包好的说明与操作步骤，按需加载 | 把"这类活该怎么干"沉淀下来，而不是每次重讲 |
| computer use | 让模型通过看屏幕截图、模拟点击键盘来操作图形界面 | 没有 API 的软件也能用 |
| 联网搜索 | 一个把查询发给搜索引擎、把结果塞回上下文的工具 | 突破训练截止时间 |

## 5.4 Agent 与聊天：差别在"后果"

聊天产品回答问题，说错了你笑一下。Agent 动手做事，做错了文件没了、配置坏了、机械臂动了。

| | 聊天 | Agent |
| --- | --- | --- |
| 输出 | 一段文字 | 文字 + 对真实系统的改动 |
| 出错代价 | 重问一次 | 可能要回滚、重装、返修 |
| 关键机制 | 提示词 | 提示词 + 权限 + 可恢复性 |

所以选 Agent 时，"多聪明"不是第一位，"权限能不能收紧、改动能不能回滚"往往更重要。

## 5.5 Harness：模型周围的那一整套

**定义：harness 是把模型的能力约束成可复现工程行为的周围系统。** 模型是发动机，harness 是底盘、变速箱、刹车和仪表盘。同一个模型，换一套 harness，产出质量可以差出好几倍。

它包含的东西比想象的多：

- **系统提示与项目指令：** `CLAUDE.md`、`AGENTS.md` ——项目约定、目录规则、禁止事项、怎么跑检查。
- **记忆：** `MEMORY.md` 之类，只留当前有效的基线和未解决风险，不写流水账。

- **任务与报告**：`agent/task/` 放需求，`agent/report/` 放每次改动的结论与验证结果。
- **git**：每个可工作的状态都是一个可回退的点。
- **测试与检查**：构建、单测、lint、手动验收清单。
- **权限**：哪些命令免问、哪些必须确认、哪些直接禁止。
- **子代理**：把可独立的工作切出去。
- **流程规矩**：一次改多少、失败怎么办、什么时候停。

教师给的四条规矩，都是有原因的：

1. **一次一改。** 一次只动一个变量，出错时才知道是谁的错。同时改五处，失败了你连从哪查都不知道。
2. **执行检查。** 改完必须跑一遍能证明它对的东西。没有检查的"完成"只是声称。
3. **权限限制。** 危险操作（删文件、改系统设置、动硬件、推远端）默认拦下问人。宁可多按一次确认。
4. **失败停止。** 同一处连续失败就停下来说清楚，不要靠反复试错硬撞——那会把上下文塞满垃圾，越试越差。

再补三条工程习惯：**模块化/解耦**（小文件、清晰边界，改动影响面小）、**报告与日志**（事后能复盘，也能给下一段会话当交接）、**git 版本管理**（一切改动的安全网）。

> 要点：harness 的核心不是"提示词写得好"，而是"让错误尽早暴露，让损失可以撤销"。

## 5.6 上下文污染：为什么聊久了会变笨

一段长对话里，你试了三种方案都失败，报错信息、废弃代码、来回的"不对，再试试"全都堆在上下文里。模型每次都要读这堆东西，并且会被它们带偏：重新提出已经否决的方案、模仿之前的错误写法、把某条过时的判断当成事实。

**典型症状**：

- 反复回到已经排除的做法上；
- 开始"修"一个根本没坏的地方；
- 忘了最初的需求，只盯着眼前那一行报错；
- 回答变短变敷衍，或者开始大段重复；
- 明明说了"不要动 X"，它还是动了。

**常见对策**：

| 手段 | 做法 |
| --- | --- |
| 开新会话 | 最有效。带上结论重开，而不是带着全部历史 |
| 交接摘要 | 写清目标、已确认的事实、已排除的方案、下一步，几行就够 |
| 任务文件 | 需求落成 `agent/task/xxx.md`，让模型读文件而不是读聊天记录 |
| 子代理隔离 | 让子代理去做翻找和试错，只把结论带回来，垃圾留在它那边 |
| 缩小任务 | 大任务拆成几个各自可验证的小步 |

这些手段的细节（压缩、检索、持久化）在第 9 章展开。

## 5.7 子代理：把可独立的工作切出去

子代理是一个被主 Agent 启动、有自己独立上下文的 Agent。它的价值在于**隔离**：翻二十个文件找一段逻辑，脏活累活干完，只回一句结论。

| 维度 | 要点 |
| --- | --- |
| 分工 | 边界清晰、能独立完成、结果可验证 |
| 协作 | 主代理负责拆解与汇总，子代理不越界 |
| 审查 | 用一个子代理检查另一个的产出，比同一个自己检查更有效 |
| 指挥 | 指令要自包含：它看不到你的对话，只有你给的那段话 |
| 隔离 | 上下文互不污染，是它最大的好处 |
| 通讯 | 回来的只是摘要，信息有损——重要的验收证据要自己复核 |

**适合用**：并行搜索多个方向、大量文件的机械整理、独立审查、隔离脏活。

**不适合用**：几行就能改完的小事（启动和交接的成本比干活还大）、危险的硬件或系统级操作（不要交给看不见的手）、需要连续手感的密集协作（来回传话丢细节）。

**成本**：每个子代理都要重新建立上下文并跑一轮，token 和时间的开销都不低；并行开多个可能成倍增加消耗。它省的是你的注意力，不是账单。

## 5.8 三种典型失败

| 现象 | 原因 | 对策 |
| --- | --- | --- |
| **幻觉**：编出不存在的函数、路径、参数、引用 | 本质是概率续写，没有内置事实核对 | 跑一遍、打开文件看、查官方文档；要求它给出可验证的依据 |
| **不守纪律**：改了没让改的、跳过检查、越权操作 | 指令太长被稀释、上下文污染、检查只写在叮嘱里而没落到程序上 | 缩小任务、明确禁止项、用权限和测试**强制**而不是靠它自觉 |
| **世界知识不足/过时**：调用已废弃的 API、按旧版本写法回答 | 训练有截止时间、无法实时更新 | 开搜索、提供当前文档或版本要求、让它先读项目里的代码再动手 |

三者的共同点：**都不是"模型坏了"，而是需要在模型外面加东西。** 这也是 harness 存在的理由。

> 要点：不要把"叮嘱得更严厉"当成修复手段。提示词能降低概率，只有程序化的检查能保证结果。

## 5.9 一次完整追踪：把按钮改成深蓝色

任务：*"把首页的购买按钮改成深蓝色。"*

```
① 观察
   读项目指令 → 知道样式在 src/styles，检查命令是 npm run check
   列目录 → 看到 src/components/、src/styles/

② 思考
   "购买按钮"对应哪个组件？颜色定义在哪？先找，不要猜。

③ 工具
   grep "立即购买" → src/components/Hero.vue 里有这个按钮
   grep "primary" src/styles → src/styles/theme.css 里有 .btn-primary { color: ... }

④ 判断（检查点）
   改动 .btn-primary 会把全站所有主按钮都改掉 —— 这是"改了没让改的"风险
   → 决定：只在 Hero.vue 里加一个局部覆盖，不动全局类

⑤ 工具：编辑 Hero.vue，加局部样式

⑥ 检查
   npm run check
```

```
    ✗ 失败：样式快照测试报错，期望值里首页按钮仍是旧颜色

⑦ 失败分支
    读报错 → 测试断言的是 .btn-primary 这个类名，局部覆盖后类名没变但快照未更新
    → 判断这是"预期变更"，更新快照前先 git diff 确认只动了这一处颜色
    → 确认 diff 只有 Hero.vue 一处，更新快照，重跑
    ✓ 通过

⑧ 汇报：改了什么文件、为什么选局部覆盖、跑了什么检查、结果如何、还有哪些没验证（比如真机视觉）
```

再看一个失败版，这是更常见的情况：Agent 图省事直接改了 `theme.css` 里的 `.btn-primary`，跑检查时样式快照测试失败——登录页、结算页的主按钮全变成深蓝了。它读到失败信息，`git diff` 看到的是一个全局文件被改，于是 `git checkout -- src/styles/theme.css` 回滚，改成局部覆盖，重跑通过。

**这个回滚之所以可行，靠的是三样东西**：git 留了旧状态、检查给出了客观的"不对"、权限允许它在受限范围内操作。缺任何一样，错误就会留到你发现为止。

## 5.10 不同 Agent 产品一览

下表只按**形态和定位**分类，不比较强弱、不排名次。产品迭代很快，具体功能和可用平台**以官方文档为准**；本课程不写版本号和价格。

| 产品 | 形态 | 定位简述 |
| --- | --- | --- |
| Claude Code | CLI + 桌面应用 | 以终端和文件为核心的编码 Agent |
| Codex | CLI / 云端 | 编码 Agent，支持在本地或远程执行任务 |
| DSH（DeepSeek Harness） | 本机 CLI | 本地运行的 harness，本课程用它处理机械性工作 |
| Grok build | CLI | 编码 Agent |
| OpenCode | 终端 TUI | 开源终端 Agent，可接不同模型 |
| Pi | CLI | 轻量终端 Agent（细节以官方文档为准） |
| Antigravity | 桌面 IDE 形态 | Agent 优先的开发环境 |
| Cursor | 桌面 IDE | 编辑器内集成的 Agent |
| Cherry Studio | 桌面客户端 | 多模型对话客户端，可接多家 API |
| WorkBuddy | 桌面应用 | 面向办公任务的 Agent 客户端（细节以官方文档为准） |

**桌面/IDE 与 CLI 的差别**，主要在这几处：

| 维度 | 桌面 / IDE | CLI |
| --- | --- | --- |
| 权限提示 | 弹窗式确认，颗粒度受界面限制 | 可按命令逐条配置允许/拒绝 |
| 文件访问 | 通常是打开的工作区 | 一般可访问整个文件系统，更灵活也更危险 |
| 终端 | 有图形终端，但 Agent 与终端交互较弱 | 天生跑在终端里，构建测试最顺手 |
| 远程/无头 | 需要图形界面 | 可在服务器、SSH 环境运行 |
| 可审计性 | 改动以 diff 面板呈现，直观 | 全程文本，容易复制、记录、进 git |

对初学者的建议：**先用桌面/IDE 形态熟悉节奏，再用 CLI 练 harness。** 两者的规矩是相通的。

## 本章小结

Agent = 模型 + 工具 + 循环 + 记忆 + 权限；它靠"观察—思考—调用—检查"的循环把事做成，而检查必须由程序给出客观答案。harness 是把这份能力约束成可复现工程行为的周围系统：指令文件、记忆、任务与报告、git、测试、权限、子代理、流程规矩。上下文会污染，所以要隔离和交接；子代理能分担但成本不低；幻觉、纪律、世界知识是三类不同的问题，对应三种不同的对策。最后一条也许最实用：**能用确定性的程序解决的，别交给 Agent。**

## 课后题

1. 观察一次你熟悉的 Agent 完成任务的全过程，把它拆成"观察—思考—工具—检查"四类步骤，指出哪一步做了真正的检查。
2. 给一个任务写一份 `AGENTS.md` 或 `CLAUDE.md` 的草稿，至少包含：项目结构、怎么跑检查、禁止做什么、一次改动多大。
3. 举一个你遇到过的"上下文污染"实例，用本章症状表对照，写出你会怎么开新会话交接。
4. 同一个任务，分别用"叮嘱它注意"和"给它一条能自动失败的测试"两种方式约束，比较结果差别。
5. （思考）什么情况下**不该**用 Agent？至少举三个例子，并说明为什么手工、写脚本或直接问人更合适。
6. （开放）如果你要给一个上初中的同学解释"Agent 和聊天的区别"，你会用哪个生活类比？这个类比在哪里会失效？